



Zenvy Dine — Complete Production Deployment & Infrastructure Guide

A beginner-friendly, production-grade deployment handbook for operating the Zenvy Dine multi-tenant restaurant QR ordering SaaS on Vercel, Firebase, and Razorpay.

Version	Audience	Environment
1.0	Founders, developers, DevOps, restaurant onboarding teams	Production and staging

Includes architecture diagrams, setup steps, security rules guidance, CI/CD, monitoring, troubleshooting, launch checklists, and a scaling roadmap.



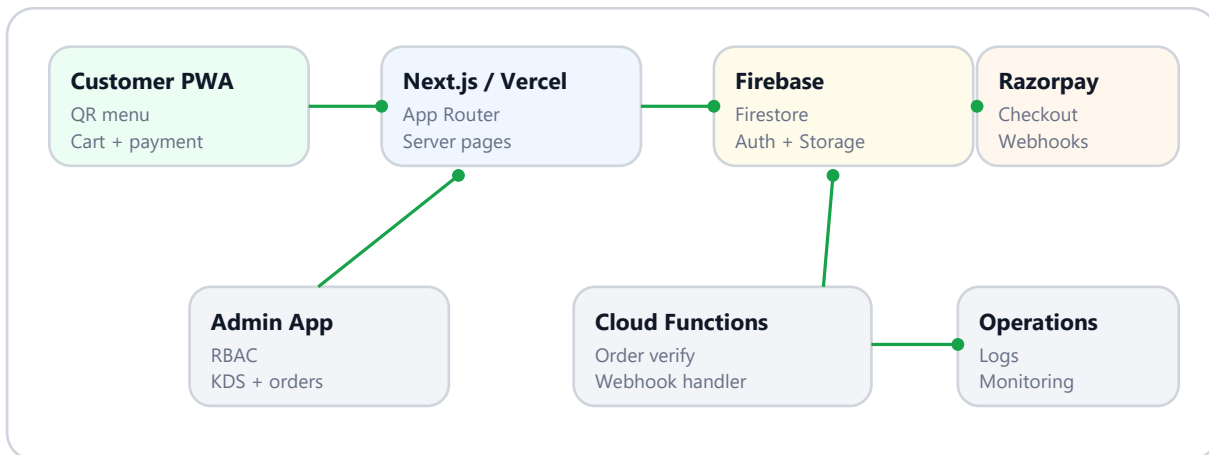
Table of Contents

Table of Contents	2
1. Introduction	4
Core architecture	4
Multi-tenant model	4
2. Local Development Setup	4
Prerequisites	4
Install and run	5
3. Firebase Project Setup	5
Step-by-step setup	5
Firebase config values	5
4. Firestore Database Setup	5
Collection structure	6
Indexes	6
Security and RBAC	6
5. Environment Variables	6
6. Razorpay Setup	7
Webhook configuration	7
Payment verification flow	7
7. Firebase Functions Deployment	8
Verify functions	8
8. Vercel Deployment	8
Preview deployments	8
9. Domain & SSL Setup	9
DNS setup	9
10. PWA Setup	9
11. Production Security	9
Security principles	9
Operational controls	10
12. Monitoring & Logging	10
13. CI/CD Pipeline	10
Rollback strategy	11
14. Performance Optimization	11
Optimization checklist	11

Firestore query efficiency	11
15. Scaling Strategy	11
16. Backup & Recovery	12
17. Troubleshooting	12
18. Deployment Checklist	12
Complete Deployment Summary	13
Production readiness checklist	13
Recommended future improvements	13
Maintenance workflow	13
Scaling roadmap	14

1. Introduction

Zenvy Dine is a multi-tenant restaurant QR ordering SaaS platform. Each restaurant gets its own branded customer ordering URL, admin dashboard, table QR codes, kitchen workflow, payment configuration, and operational settings while sharing one application codebase.



Core architecture

Layer	Primary service	Responsibility
Frontend	Next.js on Vercel	Customer menu, admin dashboard, server-rendered pages, PWA shell
Data	Firebase Firestore	Restaurants, categories, menu items, tables, orders, waiter calls, staff users
Identity	Firebase Authentication	Admin sign-in and staff identity
Backend	Firebase Cloud Functions	Razorpay order creation, payment verification, webhooks, privileged server logic
Payments	Razorpay	Online checkout, captured payment webhooks, test/live keys

Multi-tenant model

Tenant isolation is achieved with a `restaurantId` field on tenant-owned documents. Admin users also carry a `restaurantId` and role. Firestore rules validate that staff can only read or mutate records for their own restaurant.

Use one Firebase project per environment, such as `zenvy-dine-staging` and `zenvy-dine-production`. Avoid mixing staging and production tenants in the same project because billing, payment keys, and security testing become harder to reason about.

2. Local Development Setup

Prerequisites

- Node.js 20 or newer, matching the production runtime.
- Git for source control.
- Firebase CLI for rules, indexes, functions, and emulator workflows.
- A Firebase project and Razorpay test account for end-to-end testing.

```
node --version
npm --version
git --version
npm install -g firebase-tools
firebase login
```

Install and run

```
git clone <your-repository-url>
cd "Zenvy Dine"
npm install
cp .env.example .env.local
npm run validate:env
npm run dev
```

The local app typically runs at `http://localhost:3000`. If that port is already in use, Next.js may choose another port. Keep the terminal open while testing customer and admin flows.

Placeholder: Local development server running in terminal

3. Firebase Project Setup

Create separate Firebase projects for staging and production. Enable only the services required by the app and deploy rules before adding real restaurant data.

Step-by-step setup

- Open the Firebase Console and create a project named `Zenvy Dine Production`.
- Create a Web App inside the project and copy the client config values.
- Enable Firestore Database in production mode.
- Enable Firebase Authentication and turn on Email/Password and any required OAuth providers.
- Enable Cloud Storage if menu image uploads will be managed through Firebase.
- Enable Cloud Functions and select a region close to your expected restaurant users.
- Generate a service account private key from Project Settings > Service Accounts.

Firebase config values

Client Firebase config values are safe to expose as `NEXT\_PUBLIC\_\*` variables, but service account values are private and must only be stored in secure environment variable stores such as Vercel Environment Variables or Firebase Functions secrets/config.

Placeholder: Firebase project settings and web app config

4. Firestore Database Setup

Collection structure

Collection	Tenant field	Purpose
restaurants	document id / slug	Restaurant profile, branding, GST, service charge, payment settings
users	restaurantId	Admin users, role, tenant membership
menuCategories	restaurantId	Ordered menu category list
menuItems	restaurantId	Menu item catalogue, availability, images, add-ons
tables	restaurantId	Table numbers, capacities, QR URLs
orders	restaurantId	Customer orders, totals, status, payment data
waiterCalls	restaurantId	Customer service requests from table screens
payments	restaurantId	Payment intent/audit records written by Cloud Functions

Indexes

```
firebase deploy --only firestore:indexes
```

Required query shapes include orders by restaurant and `createdAt`, KDS orders by restaurant/status/createdAt, categories by restaurant/isActive/order, and waiter calls by restaurant/status/createdAt.

Security and RBAC

Roles should be explicit and minimal: `RESTAURANT\_OWNER`, `MANAGER`, `CASHIER`, `KITCHEN\_STAFF`, and `SUPER\_ADMIN`. Public customer access should only allow creating validated orders and waiter calls, never reading all orders.

Tenant isolation test: sign in as restaurant A staff and attempt to query restaurant B orders from the browser console. The request must fail with a Firestore permissions error.

5. Environment Variables

Environment variables are the boundary between code and infrastructure. Use `.env.local` only for local development. Use Vercel project settings and Firebase Functions configuration/secrets for production.

Variable	Where to obtain	Security notes
NEXT_PUBLIC_FIREBASE_API_KEY	Firebase Web App config	Public client value, not a secret
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN	Firebase Web App config	Public client value
NEXT_PUBLIC_FIREBASE_PROJECT_ID	Firebase Web App config	Must match target Firebase project
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET	Firebase Web App config	Public bucket id; storage rules still protect data
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID	Firebase Web App config	Public client value

Variable	Where to obtain	Security notes
NEXT_PUBLIC_FIREBASE_APP_ID	Firebase Web App config	Public client value
FIREBASE_CLIENT_EMAIL / FIREBASE_ADMIN_CLIENT_EMAIL	Service account JSON	Private; never commit to Git
FIREBASE_PRIVATE_KEY / FIREBASE_ADMIN_PRIVATE_KEY	Service account JSON	Private; preserve newline escaping
RAZORPAY_KEY_ID	Razorpay Dashboard API Keys	Server-side key id; use live value only in production
NEXT_PUBLIC_RAZORPAY_KEY_ID	Razorpay Dashboard API Keys	Public checkout key id
RAZORPAY_KEY_SECRET	Razorpay Dashboard API Keys	Private; rotate if exposed
RAZORPAY_WEBHOOK_SECRET	Razorpay webhook settings	Private shared secret for webhook verification

Codebase note: this repository currently uses `FIREBASE\_CLIENT\_EMAIL` and `FIREBASE\_PRIVATE\_KEY`. If your deployment platform standardizes on `FIREBASE\_ADMIN\_CLIENT\_EMAIL` and `FIREBASE\_ADMIN\_PRIVATE\_KEY`, map them consistently or update the app configuration layer before launch.

```
npm run validate:env
```

6. Razorpay Setup

- Create a Razorpay account and complete business/KYC verification.
- Use Test Mode for staging and Live Mode for production.
- Generate API keys from Razorpay Dashboard > Account & Settings > API Keys.
- Add `NEXT\_PUBLIC\_RAZORPAY\_KEY\_ID`, `RAZORPAY\_KEY\_ID`, and `RAZORPAY\_KEY\_SECRET` to the correct environment.
- Create a webhook endpoint for the deployed Firebase Function URL.

Webhook configuration

```
Webhook URL:
https://<region>-<firebase-project-id>.cloudfunctions.net/razorpayWebhook

Recommended event:
payment.captured
```

The webhook secret configured in Razorpay must match `RAZORPAY\_WEBHOOK\_SECRET` in the function environment. Signature verification is mandatory; never trust payment status sent directly from the browser.

Payment verification flow



Customer checkout creates a pending order, requests a Razorpay order from Cloud Functions, opens Razorpay Checkout, then verifies the signature server-side before marking payment as paid.

7. Firebase Functions Deployment

Cloud Functions own privileged payment operations. They use Firebase Admin SDK and Razorpay server credentials that must never be exposed to the browser.

```
cd functions
npm install
npm run build
firebase deploy --only functions
```

Verify functions

- Confirm `createRazorpayOrder` exists in Firebase Console > Functions.
- Confirm `verifyRazorpayPayment` exists and is callable.
- Confirm `razorpayWebhook` has an HTTPS URL.
- Trigger a test payment and inspect function logs.

```
firebase functions:log --only createRazorpayOrder
firebase functions:log --only razorpayWebhook
```

Screenshot placeholder: Firebase Functions dashboard with deployed functions

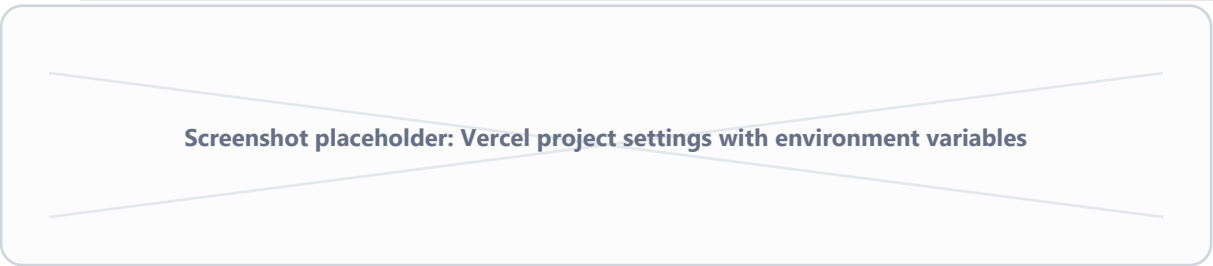
8. Vercel Deployment

- Push the repository to GitHub.
- Open Vercel and import the GitHub repository.
- Framework preset should be Next.js.
- Install command: `npm install` or `npm ci`.
- Build command: `npm run build`.
- Output settings: keep Vercel defaults for Next.js.
- Add all production environment variables.
- Deploy and inspect the build logs.

Preview deployments

Every pull request should produce a preview deployment. Use previews for UI review, smoke testing, and stakeholder approval. Do not point Razorpay live webhooks at preview URLs.

```
npm run lint
npm run build
```



9. Domain & SSL Setup

Use a clear production URL strategy. Example: `app.zenvydine.com` for the SaaS app, `demo.zenvydine.com` for client presentations, and optional custom domains for premium restaurants later.

DNS setup

Record	Host	Value	Purpose
A	@	Vercel-provided IP	Root domain
CNAME	app	cname.vercel-dns.com	Application subdomain
CNAME	www	cname.vercel-dns.com	WWW redirect

Vercel automatically provisions SSL certificates after DNS verifies. Wait for HTTPS status to become valid before sharing production URLs.

10. PWA Setup

Zenvy Dine includes a manifest, service worker, app icons, and installable app metadata. The goal is not to make Firestore fully offline-first for restaurants, but to provide a resilient app shell and installable customer/admin experience.

Asset	Path	Purpose
Manifest	/manifest.json	Name, theme color, display mode, icon list
Service worker	/sw.js	Caches app shell and static assets
192 icon	/icons/icon-192x192.png	Android/Chrome install icon
512 icon	/icons/icon-512x512.png	High-resolution install icon
Favicon	/favicon.ico	Browser tab identity

```
curl https://<domain>/manifest.json
curl https://<domain>/icons/icon-512x512.png
```

11. Production Security

Security principles

- Never trust client-computed totals without server-side validation for high-risk payment paths.
- Never expose service account JSON or Razorpay secret in public variables.

- Keep tenant checks in Firestore rules and server-side admin operations.
- Validate webhook signatures before updating payment state.
- Use staging projects for destructive testing.

Operational controls

Risk	Control
Cross-tenant data leakage	All tenant documents include `restaurantId`; rules compare with user claim/document
Public order spam	Validate order shape; add App Check/rate limiting for production
Payment spoofing	Use Razorpay signature verification and webhook secret
Secret leakage	Use Vercel encrypted env vars; rotate keys after exposure
Over-privileged staff	Use role-specific access and periodic staff audits

12. Monitoring & Logging

- Use Vercel build/runtime logs for frontend deployment and server-rendering issues.
- Use Firebase Functions logs for payment creation, verification, and webhook errors.
- Use Firestore usage dashboards to monitor reads, writes, deletes, and index growth.
- Add uptime monitoring for the production root URL and a demo restaurant menu URL.
- Add error tracking before onboarding paying restaurants.

```
firebase functions:log
firebase firestore:indexes
vercel logs <deployment-url>
```

Screenshot placeholder: Monitoring dashboard: uptime, error rate, function failures

13. CI/CD Pipeline

CI/CD prevents broken builds from reaching production. The repository includes a GitHub Actions workflow that installs dependencies, runs ESLint, and runs a production build.

```
name: CI
on:
  push:
    branches: [main]
  pull_request:
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
```

```
- run: npm ci
- run: npm run lint
- run: npm run build
```

Rollback strategy

- Use Vercel's previous deployment promotion for frontend rollback.
- Keep Firebase rules and indexes versioned in Git.
- Deploy Cloud Functions from tagged releases.
- Record every production environment variable change in a release note.

14. Performance Optimization

Optimization checklist

Check	Launch item
[]	Use `next/image` for menu item and logo images.
[]	Use appropriately sized remote images and configured image domains.
[]	Avoid unbounded Firestore collection reads; always filter by `restaurantId`.
[]	Keep realtime listeners limited to active operational screens.
[]	Use Firestore indexes for combined filters and sorting.
[]	Lazy-load non-critical client UI where possible.
[]	Run Lighthouse against production URLs after every major UI change.

Firestore query efficiency

Realtime listeners can become expensive when restaurants scale. KDS and order screens should query only the current restaurant and relevant statuses. Archive or paginate old completed orders.

```
npm run build
npx lighthouse https://<production-url> --only-categories=performance,accessibility,seo,best-practices
```

15. Scaling Strategy

Stage	Architecture	Actions
1-10 restaurants	Single Firebase project	Tenant isolation, manual onboarding, basic logs
10-100 restaurants	Staging + production projects	Automated seeding, staff roles, monitoring, support playbooks
100-500 restaurants	Operational automation	Data retention, analytics aggregation, stronger rate limiting
500+ restaurants	Segmented infrastructure	Regional projects, queue-backed notifications, warehouse analytics

Firestore scales well when document paths and indexes are designed around query patterns. Avoid global realtime dashboards that read all restaurants at once. Aggregate analytics into separate documents instead of scanning orders repeatedly.

16. Backup & Recovery

- Schedule Firestore exports to a protected Google Cloud Storage bucket.
- Keep Firebase rules, indexes, and functions in Git.
- Use Vercel deployment history for frontend rollback.
- Document manual recovery steps for restaurant configuration and menu data.
- Test restore procedures quarterly in a staging project.

```
gcloud firestore export gs://<backup-bucket>/firestore/$(date +%Y-%m-%d)
gcloud firestore import gs://<backup-bucket>/firestore/<export-folder>
```

For payment records, preserve Razorpay dashboard exports and webhook logs. Financial reconciliation should not depend only on Firestore order documents.

17. Troubleshooting

Issue	Likely cause	Fix
Firebase auth errors	Provider disabled or wrong auth domain	Enable provider and verify `NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN`
Firestore permission denied	Missing role, wrong restaurantId, or strict rules	Inspect user document and security rule path
Razorpay checkout opens but payment not paid	Verification function failed	Check function logs and key/secret pair
Webhook failures	Wrong webhook secret or URL	Recreate webhook and compare `RAZORPAY_WEBHOOK_SECRET`
Vercel build fails	TypeScript or missing env var	Run `npm run build` locally and validate env
Hydration warnings	Browser-only code rendered on server	Move browser APIs into client components/effects
PWA icons 404	Missing `/public/icons` files	Verify icon paths and manifest icons
Image warnings	CSS changed only one dimension	Use `width:auto` or `height:auto`; preserve ratio

When troubleshooting production, always collect the exact URL, deployment id, Firebase project id, restaurant slug, user role, browser console output, and relevant function logs before changing code.

18. Deployment Checklist

Check	Launch item
[]	Production Firebase project created and separate from staging.
[]	Firestore, Authentication, Storage, and Functions enabled.
[]	Firestore rules deployed and tenant isolation manually tested.
[]	Firestore indexes deployed.

☐	Vercel production environment variables added.
☐	Firebase Functions secrets/environment configured.
☐	Razorpay live keys configured only in production.
☐	Razorpay webhook points to production function URL.
☐	Custom domain connected and HTTPS verified.
☐	PWA manifest, icons, favicon, and service worker return HTTP 200.
☐	Mobile customer order flow tested on at least two device sizes.
☐	Admin live orders, KDS, waiter calls, QR printing, and receipts tested.
☐	Lighthouse performance/accessibility/best-practices/SEO reviewed.
☐	Rollback plan documented.
☐	Monitoring and support alert channels configured.

Complete Deployment Summary

A production Zenvy Dine deployment consists of a Next.js frontend on Vercel, Firebase Firestore/Auth/Storage/Functions for data and privileged server logic, and Razorpay for checkout and payment verification. The operational heart of the system is tenant isolation through `restaurantId`, role-based admin access, and strongly protected payment verification.

Production readiness checklist

Check	Launch item
☐	Build and lint pass.
☐	Environment validation passes.
☐	All public assets return 200.
☐	Payment test succeeds in Razorpay test mode.
☐	Firestore security rules reject cross-tenant reads.
☐	Admin and customer flows work on mobile.
☐	Backups and rollback plan are documented.

Recommended future improvements

- Add Firebase App Check and rate limiting for public order and waiter-call writes.
- Move WhatsApp notifications from link-based flow to Meta/Twilio server-side messaging.
- Add automated Firestore rules tests in CI.
- Add end-to-end browser tests for customer ordering, KDS, and receipts.
- Add structured error tracking such as Sentry or a Vercel/Firebase observability stack.

Maintenance workflow

- Weekly: inspect Vercel and Firebase logs, review failed payments, verify uptime checks.

- Monthly: rotate non-critical test keys, review staff access, export Firestore backup reports.
- Quarterly: run restore drill, Lighthouse audit, security rule review, and dependency updates.

Scaling roadmap

Phase 1 focuses on reliable onboarding and tenant isolation. Phase 2 adds automated setup, stronger monitoring, and notification infrastructure. Phase 3 adds analytics aggregation, cost controls, and regional segmentation for larger restaurant portfolios.